AgriSense: Viva Study Guide

An Intelligent Crop Recommendation System · Jasleen Kaur, Anand Kumar Mishra, Aditya Jaiswal · Supervisor: Dr. Kalicharan · Prepared 16 Oct 2026

Read this first. Your slides promise a few things the built project does not do yet. The examiner will compare the two, so be honest about them (table in section 1) and use the answers in section 8.

1. Slides versus what is built

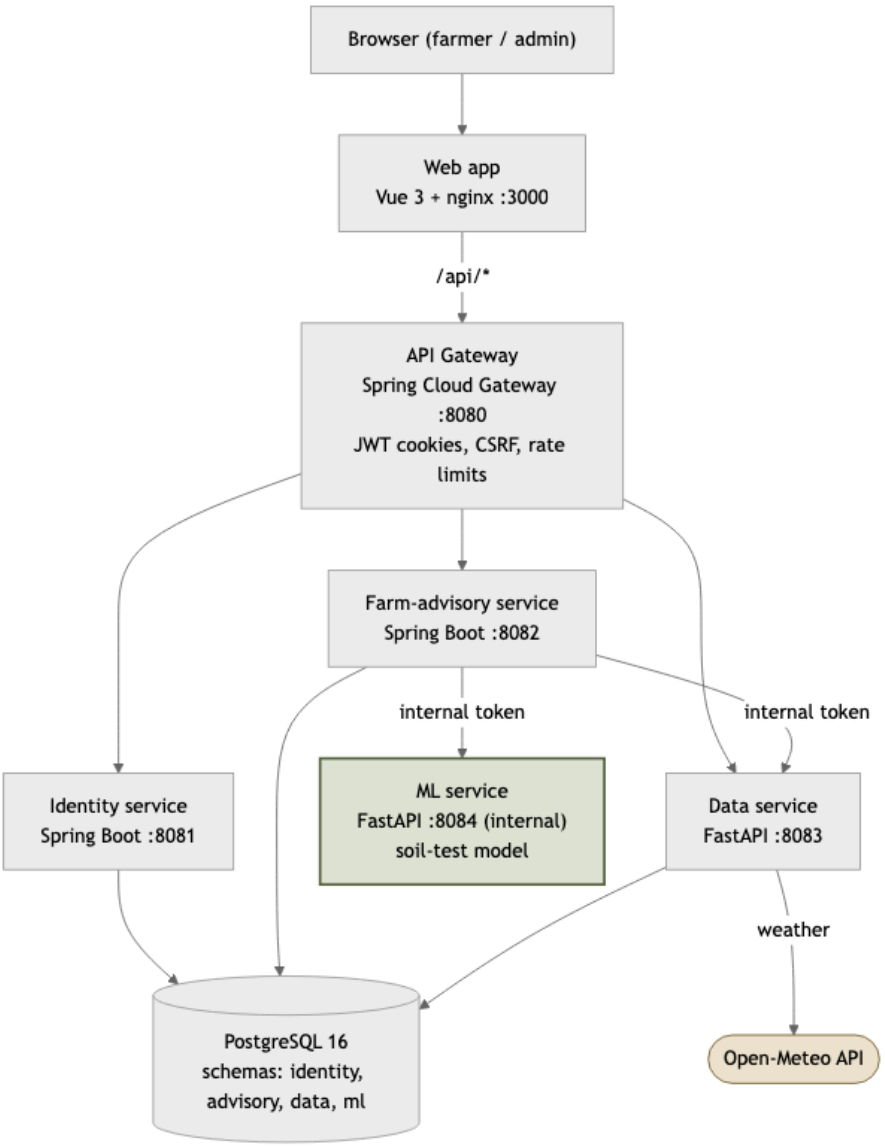
Slides say	What exists	What to say
Stacking ensemble (Random Forest, XGBoost, CatBoost), Optuna tuning	One Random Forest (100 trees). No stacking or Optuna in the shipped code.	These are experiments and future work.
SHAP and LIME	SHAP implemented and verified. No LIME.	SHAP done, LIME planned.
Fertilizer type and quantity	Not built. The app shows "not available yet".	Deferred: needs a verified source and a soil test.
Authentic government data, no synthetic data	Regional guide uses real historical district records. The soil-test dataset looks synthetic.	Say so yourself; it is more credible.
FastAPI or Flask backend; mobile and web	Spring Boot plus FastAPI services; responsive web app and installable PWA.	The design evolved for security and separation.
Accuracy above 96%	99.55% held-out and 5-fold, dataset-internal only.	Always add "not real-farm accuracy".

2. What the project is

- **Problem:** farmers often choose crops by habit while climate and soil change; many ML tools are black boxes that farmers do not trust (SDG 2, Zero Hunger).
- **Solution:** a web app that gives district-level crop guidance with transparent reasoning, plus weather.
- **Main feature, Regional Crop Guide:** which crops were historically grown in your district this season, ranked by share of cropped area.
- **Second feature, soil-test prediction:** your ML model ranks 22 crops from 7 values and shows the SHAP factors. It is labelled experimental.
- **Also:** weather and 7-day forecast, action plans, alerts, government scheme links, English and Hindi, installable PWA, admin pages.
- **Process:** Agile-iterative, four 2-month phases: data, preprocessing, training, recommendation, explainability, dashboard and deployment.

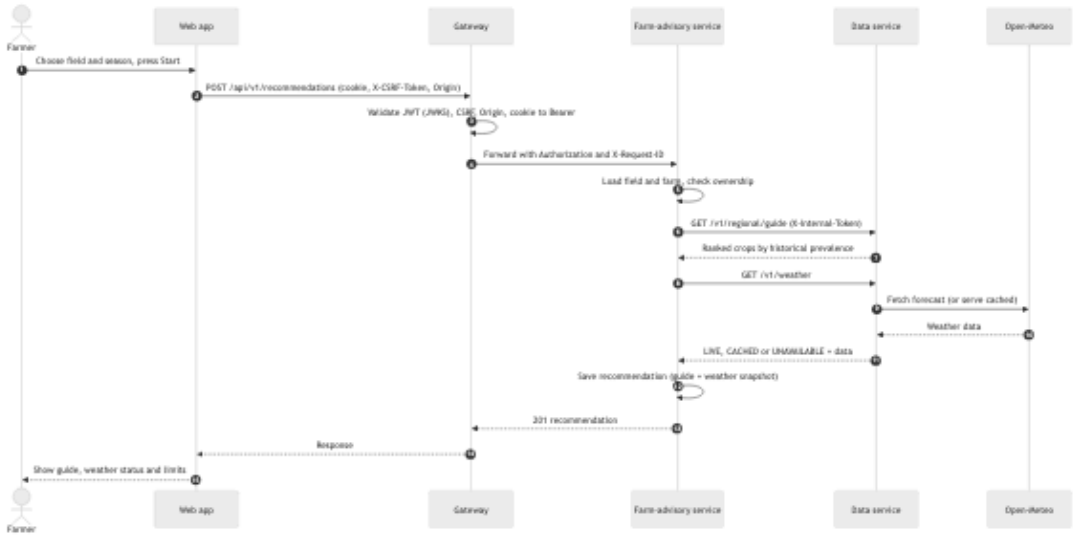
3. Architecture

Flow: **browser, Vue 3 web app (nginx), API gateway, one of three services, PostgreSQL.** The ML service is internal only.



Component	Technology	Job
Web app	Vue 3, TypeScript, Tailwind, nginx	All screens, English and Hindi, PWA
Gateway	Spring Cloud Gateway :8080	Single entry; JWT cookies, CSRF, origin check, rate limits
Identity service	Spring Boot :8081	Accounts, Argon2id, RS256 JWT, refresh-token rotation, roles
Farm-advisory service	Spring Boot :8082	Farms, fields, recommendations, plans, alerts, feedback, prediction endpoint
Data service	FastAPI :8083	522 districts, crop-guide engine, Open-Meteo weather
ML service	FastAPI :8084 (internal)	Soil-test model with SHAP
Database	PostgreSQL 16	One schema per service

Request walkthrough: Regional Crop Guide



Say it in words: the browser sends a request with cookies; the gateway validates the token, CSRF and origin and passes it on; farm-advisory loads the field, asks the data service for the guide and weather, saves the result and returns it.

4. Why these technology choices

Why microservices, and why not all Python?

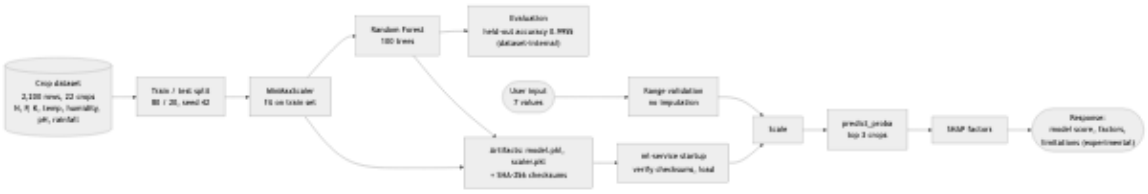
- **Isolation:** the ML service loads pickle files, which can run code, so it sits internal-only, away from users.
- **Right tool per job:** Python for ML and data; Java and Spring for security-critical identity and gateway.
- **Independent failure:** weather or ML can be down while login and the guide still work.
- **Team fit and testing:** separate services map to separate roles and have their own contracts and tests.
- **Honest trade-off:** more operational overhead; a modular monolith would also work at this scale. Docker Compose keeps it to one command.

Choice	Why
Vue 3	Lighter and simpler than React or Angular; good TypeScript, Pinia, Router, i18n support; small build.
Spring Boot	Mature security, validation and testing; standard gateway pattern; enterprise practice.
FastAPI	Native Python, so scikit-learn and SHAP work directly; fast; automatic validation and docs.
PostgreSQL	Relational, reliable, free; JSON columns for snapshots; schema per service.
Docker Compose	Whole stack with one command; reproducible for the team and evaluators.

Feasibility (from your slide)

- **Technical:** mature open-source stack; published work reports 95 to 99 percent on the same 7 features; it already runs end to end.
- **Economic:** free data, free tools, no sensors in this phase.
- **Operational:** the 7 inputs are already printed on the Soil Health Card.
- **Schedule:** 8 months in four 2-month phases, three members in parallel.

5. The ML model



- **Model:** Random Forest, 100 trees, 7 inputs (N, P, K, temperature, humidity, pH, rainfall), 22 crops.
- **Data:** 2,200 rows, 100 per crop. Split 80/20, seed 42. MinMax scaler fitted on the training set.
- **Results:** 99.55% on held-out data and in 5-fold cross-validation. Weakest classes: rice and jute (F1 about 0.96). With noisy inputs: 96% (small noise), 81% (moderate).
- **Output:** top 3 crops with a *model score* (vote share, not a probability; calibration error 0.039) and SHAP factors (additivity error 4e-16).
- **Why Random Forest:** excellent on small tabular data, little tuning, millisecond predictions, exact SHAP for trees; deep learning would overfit 2,200 rows and be harder to explain; XGBoost and CatBoost would give similar accuracy with more tuning.

A strength to tell: we found the shipped scaler was defective (fitted on one sample, only 7.2% agreement with labels). We rebuilt it (accuracy back to 99.55%) and now verify file checksums at load.

Honest limits: data looks synthetic with generated-looking ranges; rainfall averaging period undocumented; 22 crops only (no wheat, sugarcane, mustard, soybean, potato). So the model is labelled experimental and never mixed into the Regional Guide.

Regional Crop Guide engine



Recent years weigh more (half-life 1 year); thin districts blend in the state pattern (shrinkage k = 2). Back-test (history to 2012, test 2013-2014): top-3 overlap with real plantings **0.811** versus **0.572** for a state-popularity baseline. This measures agreement with history, not agronomic suitability.

6. Figma

- **File:** "AgriSense Website", private draft: https://www.figma.com/design/Hjf66gqkHJTxfJzkoap53V
- **Pages (7):** landing (desktop and mobile), sign in, overview, crop advice, crop prediction, weather.
- **Design system:** forest-green, cream and ochre palette; Newsreader serif headings, Inter body text; rounded cards, soft shadows.

- **Principles:** honest labelling, accessibility (WCAG 2 A/AA, tested), responsive, English and Hindi.
- **Be honest:** the file was generated from the running site, so it documents the final design. Early design references are the V9 and V14 HTML files in the project folder. Layers are not yet reusable components.

7. Rubric mapping (50 marks)

Criterion (marks)	Show
1. Quantified functional specs (6)	Requirement slide with numbers; API contracts in `contracts/`; product rules in conventions.
2. Framework / SDLC model (6)	Agile-iterative phases; microservices rationale; ADRs 0001 to 0005.
3. Latest design tools (6)	Figma, Vue 3, Tailwind v4, Vite, Docker Compose, OpenAPI, Playwright, axe, SHAP, Mermaid diagrams.
4. Appropriateness of design (6)	Architecture, sequence, ER and DFD diagrams; security design; honest labelling rules.
5. Functionality matches design (6)	Live demo; test record (about 150 backend tests, 61 web tests, browser suites).
6 and 7. Oral and written (5 + 5)	The one-minute pitch below; this guide, the team PDF and the README.

8. Questions and answers

Q1. Why microservices and not just Python?

Four reasons. (1) Isolation: the ML service loads pickle files, which can execute code, so it is internal-only and separate from anything users can reach. (2) Right tool per job: Python for ML and data work, Java/Spring for security-critical identity and gateway code. (3) Independent failure: if weather or ML is down, login and the Regional Guide still work. (4) Testability: each service has its own API contract and tests. Honest trade-off: it adds operational overhead and a modular monolith could also work at this scale; we chose microservices to demonstrate clear service boundaries and security isolation, and Docker Compose keeps it to one command.

Q2. Why Vue 3?

Lighter and simpler than React or Angular, with strong TypeScript support, Pinia for state, Vue Router and vue-i18n for English/Hindi. The learning curve suits a small team and the build is small and fast.

Q3. Why Spring Boot?

Production-grade, mature security (JWT, CSRF, rate limiting), strong validation and testing support, and the Spring Cloud Gateway gives a standard API-gateway pattern. It also demonstrates enterprise patterns.

Q4. Why FastAPI for the ML and data services?

It is native Python, so scikit-learn, SHAP and pandas are used directly with no bridge. It is fast, validates inputs automatically and generates API documentation.

Q5. Why PostgreSQL, and why one database with several schemas?

Relational, reliable and free, with JSON columns for stored snapshots. One schema per service keeps data ownership separate (a service only uses its own schema) while keeping local setup simple. In production each could become its own database.

Q6. What is an API gateway and why do you need one?

A single entry point in front of all services. It validates the login token, checks CSRF and origin, applies rate limits and routes requests. Services behind it never face the internet, so security rules live in one place.

Q7. How is authentication done?

Argon2id password hashing. On login, identity issues a 15-minute RS256 access JWT and a rotating refresh token, stored in HttpOnly cookies. A CSRF token is double-submitted for state-changing requests. Reusing an old refresh token revokes the whole token family.

Q8. Why a Random Forest and not deep learning, XGBoost or CatBoost?

The data is small tabular data (2,200 rows, 7 features). Random Forest reaches very high accuracy there, needs little tuning, predicts in milliseconds, and SHAP's TreeExplainer is exact for tree models, which serves our transparency goal. Deep learning would overfit and be harder to explain. XGBoost and CatBoost would likely give similar accuracy with more tuning effort; they are listed as future experiments.

Q9. Is 99.55 percent accuracy real?

It is real for this dataset (held-out and 5-fold cross-validation), but it is dataset-internal. The data looks synthetic and has tight, generated-looking ranges, so it does not prove accuracy on real farms. That is why the Regional Crop Guide is the main feature and the model is labelled experimental.

Q10. What is SHAP and what is LIME? Which do you use?

Both explain model predictions. SHAP gives each feature a contribution to a prediction and, for tree models, is exact. LIME explains one prediction by fitting a simple local model around it. We implemented SHAP (additivity verified to 4e-16). LIME is planned.

Q11. What is the Regional Crop Guide and how is it calculated?

For a district and season it ranks crops by historical prevalence: the share of recorded cropped area each crop held. Recent years weigh more (half-life of 1 year), and when a district has little data it blends in the state pattern (shrinkage k = 2). On a back-test (history to 2012, test on 2013-2014) its top-3 overlap with what was really planted was 0.811, versus 0.572 for a state-popularity baseline.

Q12. Why do you say 'model score' and not probability?

A Random Forest vote fraction is not a calibrated probability (our calibration error was 0.039). Calling it a probability would mislead farmers, so the interface says 'model score, not a probability'.

Q13. What problems did you find in the original model files?

The shipped scaler had been fitted on a single sample, so predictions agreed with the labels only 7.2 percent of the time. We reconstructed the scaler as in the team notebook (accuracy back to 99.55 percent) and now verify file checksums at load. The requirements file also pinned the wrong scikit-learn version.

Q14. What happens if the ML or weather service is down?

Weather is shown as Live, Cached or Unavailable with the source and time, and the guide still works. If the ML service is down, only the prediction page shows an error and everything else keeps working.

Q15. What are your functional and non-functional requirements?

Functional: input acquisition with range validation, crop prediction (22 classes, ranked), explainability output, dashboard and history, accounts and admin. Non-functional (each with a number): under 2 s per request, accuracy 96 percent minimum, 7 inputs on one screen, low-confidence flagged, HTTPS and token access, modular services, documented on GitHub.

Q16. What is your SDLC model?

Agile and iterative with four 2-month phases over 8 months, three members in parallel (data and model, backend and services, frontend and QA). Decisions are recorded as architecture decision records (ADR 0001 to 0005).

Q17. How did you test it?

About 150 backend tests (gateway 30, identity 32, farm-advisory 45, data 32, ML 9), 61 web unit tests, browser tests against the real running stack and with mocks, and accessibility checks (WCAG 2 A and AA). The testing record is in docs/testing.md.

Q18. What about privacy?

Only an email and password hash are stored for an account. GPS coordinates are sent once to find the nearest district and are not stored. Soil values are optional. Prediction inputs are not stored.

Q19. Why not a mobile app?

A responsive web app that can be installed as a PWA works on any phone without app-store distribution. Real-device testing is still pending, so say it was only tested with Chromium device emulation.

Q20. What is novel about this project?

Honest, transparent advice: a regional guide backed by a back-test, an explainable experimental model, visible data status (live, cached, unavailable), and security-first microservices. It also avoids overclaiming: no yield, price or fertilizer numbers without a verified source.

Q21. What are the limitations and future work?

Limitations: the model's dataset looks synthetic and rainfall period is undocumented; only 22 crops; crop records end in 2014; dataset licences are unverified; Hindi text is a draft; no real-device or deployed-server testing. Future work: LIME, fertilizer module with a verified source, stacking ensemble with Optuna tuning, real farm data, deployment and email for password reset.

9. One-minute pitch

"AgriSense helps farmers decide what to grow. Farmers often pick crops by habit while climate and soil change, and many ML tools are black boxes they cannot trust. Our main feature is the Regional Crop Guide, which shows what was historically grown in your district this season, backed by a back-test that beat a state-level baseline. Alongside it, an experimental Random Forest model ranks 22 crops from seven soil and weather values, with SHAP explaining each result and clear labels that its score is not a probability. The system is a set of microservices behind a secured gateway: a Vue 3 web app, Spring Boot identity and advisory services, and FastAPI data and ML services, on PostgreSQL, all started with Docker Compose. We designed it in Figma, tested it with about 150 backend tests plus browser and accessibility checks, and we are open about limits: the model data looks synthetic, so the guide leads. Next we plan LIME, a fertilizer module with a verified source, and real farm data."

10. Demo checklist

- Start: `docker compose up -d`, open http://localhost:3000.
- Sign in with the account you created, show Overview (weather status badge), Crop Advice, then Crop Prediction.
- Prediction demo values: N 90, P 42, K 43, 20.9 °C, 82%, pH 6.5, 203 mm (Rice). Show an out-of-range pH to demonstrate validation.
- Show the architecture slide, then the Figma file and the diagrams in `docs/diagrams/png`.